

Motivation: From RNNs/LSTMs → Self-Attention

1. Sequential Nature of RNNs and LSTMs

How RNNs Work

In Recurrent Neural Networks (RNNs) and LSTMs:

- Input is processed one word at a time
- Each step depends on the previous hidden state

Mathematical Flow

At time step t :

$$h_t = f(x_t, h_{t-1})$$

Meaning:

- The current hidden state depends on:
 - Current input x_t
 - Previous hidden state h_{t-1}

Key Property

Computation is sequential.

Example

Sentence:

“The cat sat on the mat”

Processing happens as:

- Step 1 → “The”
- Step 2 → “cat” (depends on “The”)
- Step 3 → “sat” (depends on previous states)
- ...

Therefore, all words cannot be processed simultaneously.

2. Limitations of Sequential Processing

2.1 No Parallelization

- Cannot fully utilize GPUs
- Training is slow

Reason:

- Must compute h_{t-1} before h_t

2.2 Long-Range Dependency Problem

Even with LSTMs:

- Earlier information fades
- Difficult to connect distant words

Example:

“The book that I bought yesterday from the store was amazing”

- “book” and “was” are far apart
- The model struggles to relate them effectively

2.3 Information Bottleneck

- Information is compressed step-by-step
- Earlier details may be lost

2.4 Gradient Issues

- Vanishing gradients (reduced in LSTM but still present)
- Learning long dependencies remains difficult

3. Key Insight That Led to Self-Attention

Core idea: Instead of processing words sequentially, allow every word to directly interact with every other word.

4. Introduction of Self-Attention

What Self-Attention Does

- Takes the entire sentence as input
- Computes relationships between all word pairs simultaneously

Conceptual Shift

RNN/LSTM	Self-Attention
Sequential	Parallel
Step-by-step	All words together
Local memory	Global context

5. How Self-Attention Solves These Problems

5.1 Parallel Processing

- All words are processed at once
- Efficient utilization of GPUs
- Faster training

5.2 Direct Word-to-Word Connections

Each word can directly attend to any other word.

Example:

“The bank approved the loan”

- “bank” directly attends to “loan”
- No need for intermediate steps

5.3 Better Long-Range Dependency Handling

- No distance limitation
- Every word is directly accessible

5.4 No Hidden State Bottleneck

- No single compressed representation
- Information flows across the sequence freely

6. Why Positional Encoding is Needed

New Problem Introduced by Self-Attention

Since all words are processed simultaneously:

- The model loses information about word order

Example

The following sentences appear identical to the model:

- “cat sat on mat”
- “mat sat on cat”

Solution

Introduce positional encoding so the model can learn:

- Which word comes first
- Which word comes later

Final Summary

RNNs/LSTMs

- Sequential processing
- Slow due to lack of parallelization
- Difficulty with long-range dependencies

Self-Attention

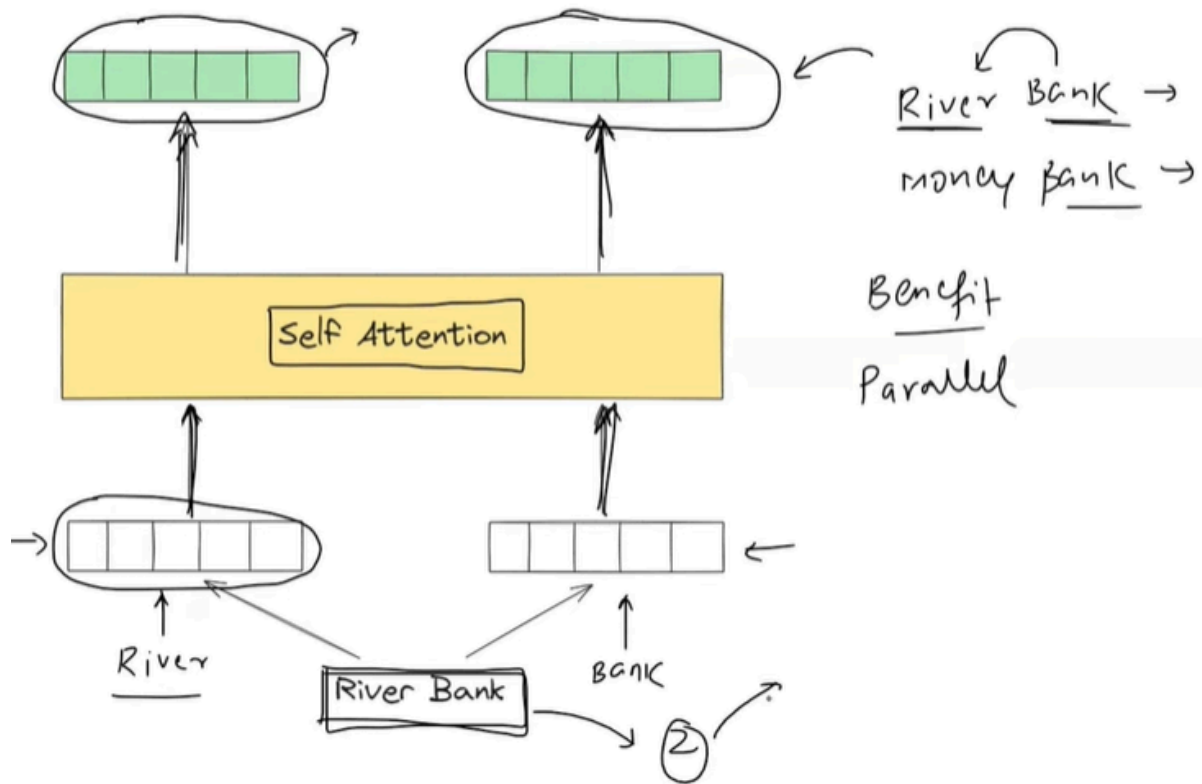
- Processes all words simultaneously
- Captures global relationships
- Removes sequential bottleneck

Trade-off

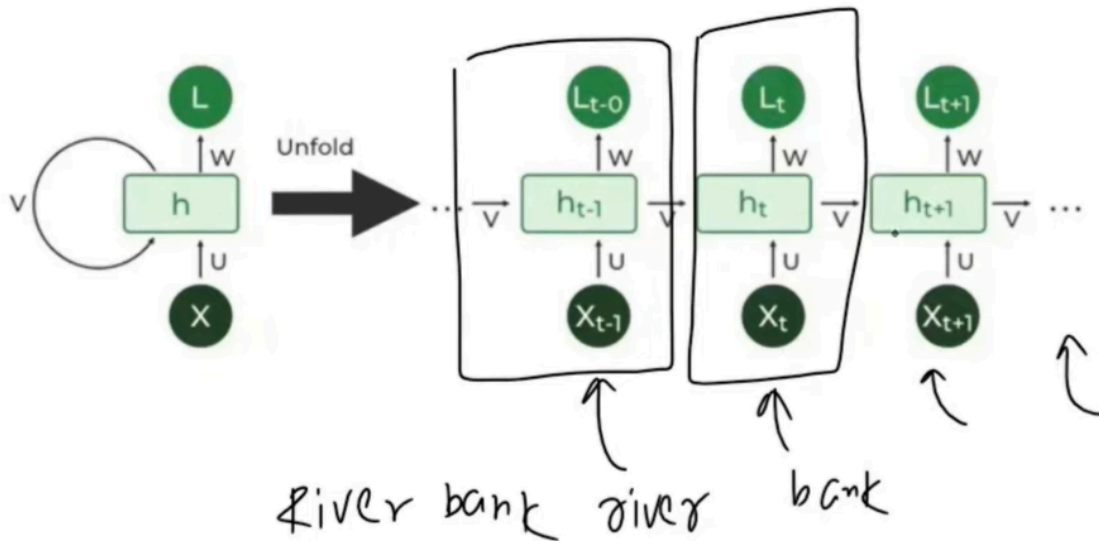
- Loses positional information
- Addressed using positional encoding

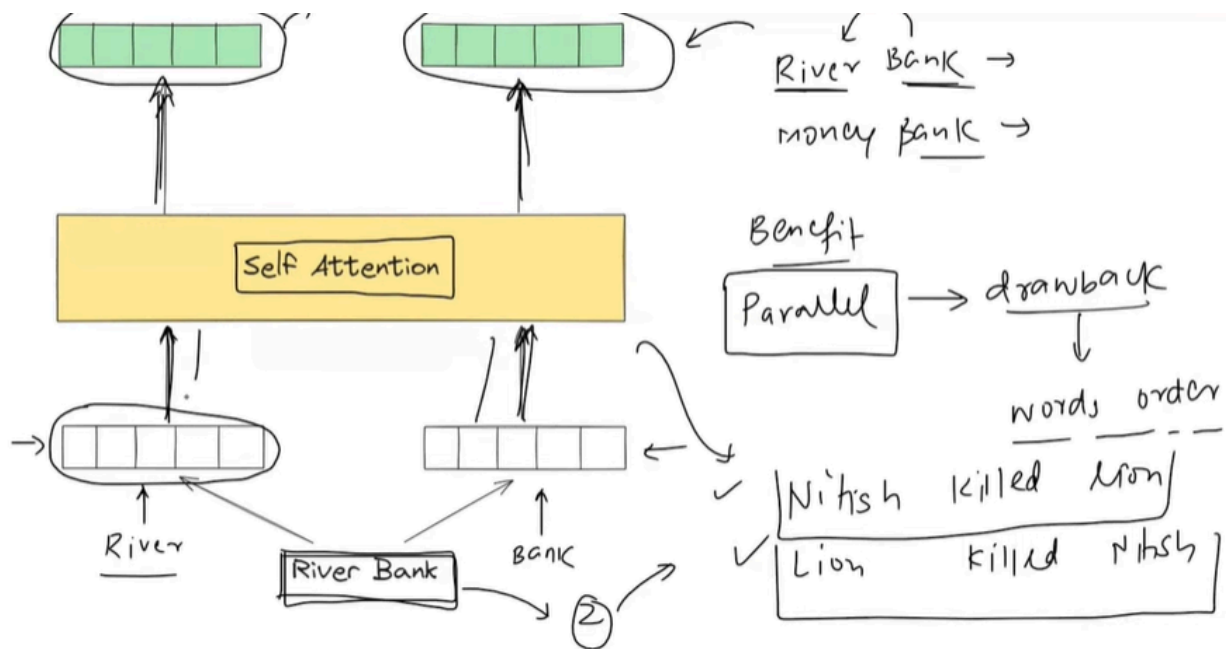
One-Line Insight

Self-attention replaces sequential dependency with direct, parallel, global interaction between words.



RNN → sequential





Self attention loses order of words in a sentence.

Positional Encoding in Transformers - Detailed Notes

1. Problem: Why Positional Encoding Was Needed

Self-Attention Limitation

In the Transformer architecture (from the paper **Attention Is All You Need**):

- Input is processed **in parallel**
- Unlike RNNs/LSTMs, there is **no inherent notion of sequence order**

This means:

The model does **not know which word comes first, second, or last**

Example

Sentence:

"Nitesh killed the lion"

Self-attention sees embeddings like:

E(Nitesh), E(killed), E(the), E(lion)

But it **cannot distinguish**:

- "lion killed Nitesh"
- "Nitesh killed lion"

Because **order is lost**

2. Naive Solution (Rejected Approach)

Idea:

Append position index to each word embedding:

Nitesh → [embedding | 1]

killed → [embedding | 2]

the → [embedding | 3]

lion → [embedding | 4]

Problems with This Approach

1. Unbounded Values

- For long sequences (e.g., PDFs, books), positions become very large:
1, 2, 3, ..., 10,000+
- Neural networks prefer **bounded inputs**

Causes:

- Training instability
- Difficult gradient updates (backpropagation issues)

2. Discrete Nature

- Position indices are **integers (1, 2, 3, ...)**

Problem:

- Neural networks work better with **continuous values**
- Discrete jumps → harder optimization

3. No Relative Position Awareness

- Model cannot easily learn relationships like:

- "word A is 2 steps ahead of word B"

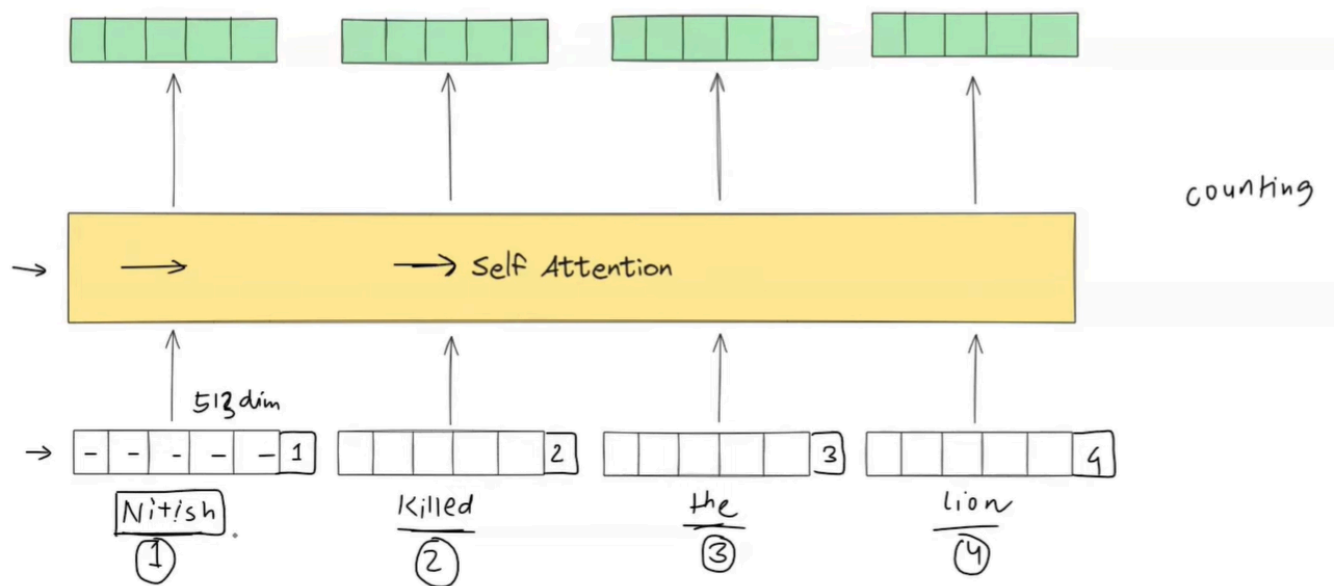
Absolute numbers don't encode:

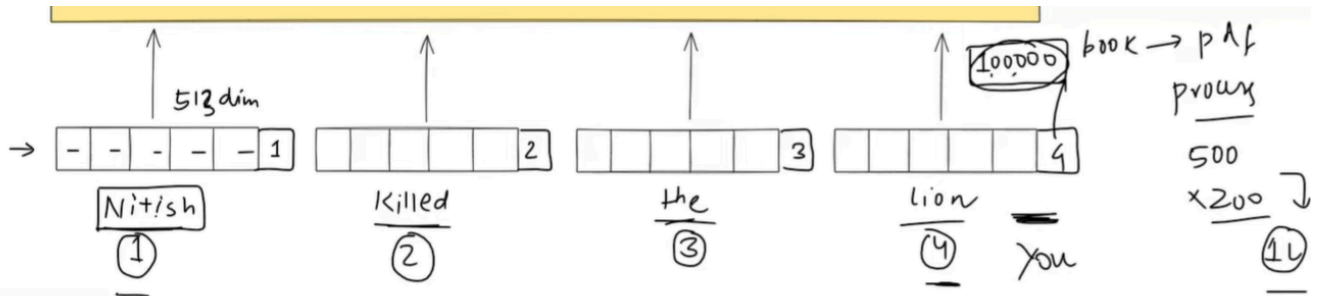
- Distance
- Patterns of spacing

3. Design Requirements for a Better Solution

We need a positional representation that is:

Property	Why Needed
Bounded	Stable training
Continuous	Smooth gradients
Periodic	Capture relative positions
Generalizable	Works for longer sequences





Problem

1) unbounded $\rightarrow$ upper limit (huge)

-1 to 1

big number

backprop

NN architecture

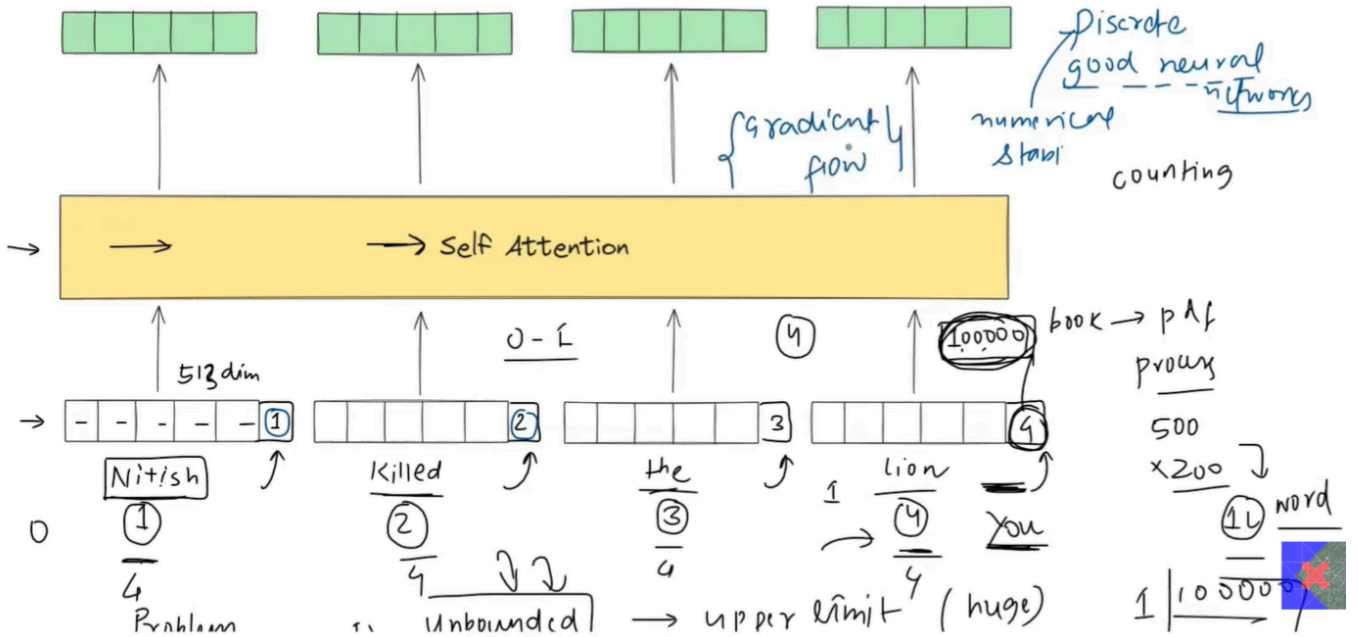
unstable

gradient unstable

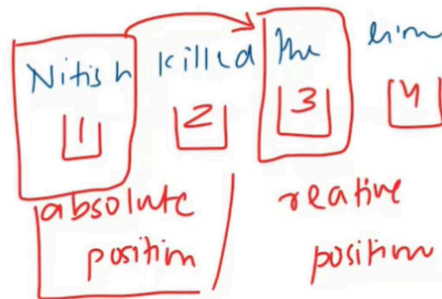
23 May 2024 15:37

smooth handling, continue

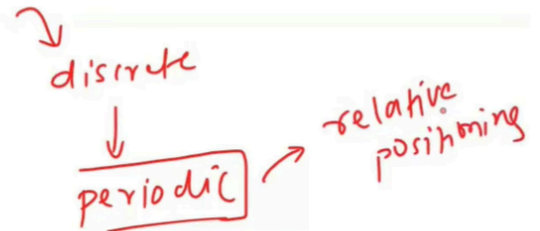
Problem 2



Problem 3 → can't relative positioning



$$3 - 1 = 2$$



Problem

→ unbounded (bounded) ✓✓

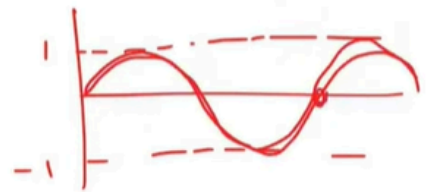
→ discrete (continuous) → function →

→ relative (periodic) position

trigonometric

↓

sine



4. Key Insight: Use Mathematical Functions

We need a function that is:

- Smooth
- Repeating (pattern-based)
- Within fixed range

That leads us to:

Trigonometric Functions

- Sine (sin)
- Cosine (cos)

Bounded: between -1 and 1

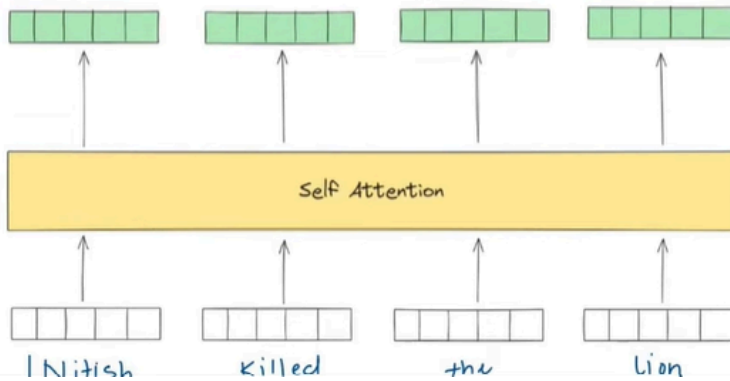
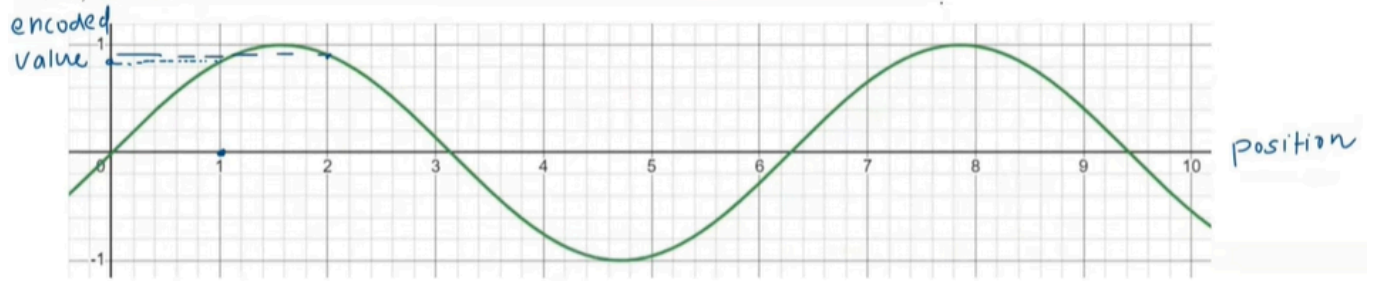
Continuous

Periodic

Infinite extension

The sine function as a solution

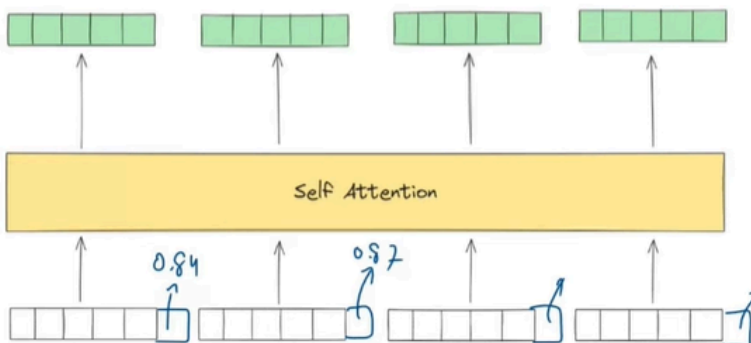
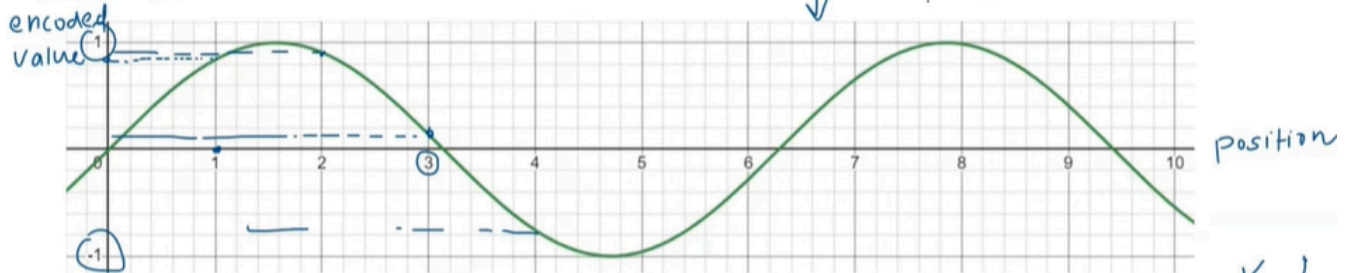
23 May 2024 17:17



$y = \sin(\text{position})$
 $y = \sin(1) = 0.84$
 $y = \sin(2) = \dots$

The sine function as a solution

23 May 2024 17:17



$y = \sin(\text{position})$
 $y = \sin(1) = 0.84$
 $y = \sin(2) = 0.87$
 $y = \sin(3) = 0.14$

- ✓ 1
- ✓ 2
- ✓ 3

Limitation of Using Only Sine for Positional Encoding

1. Initial Idea Recap

We decided to use a function that is:

- Bounded
- Continuous
- Periodic

So we considered:

$$PE(pos) = \sin(pos)$$

2. Problem: Periodicity Causes Collisions

Key Property of Sine

The sine function is **periodic**:

$$\sin(x) = \sin(x + 2\pi k)$$

This means values **repeat after every cycle**

What Goes Wrong?

Different positions can map to the **same value**.

Example:

- Position A: $pos=1$
- Position B: $pos=1+2\pi$
- $\sin(1) = \sin(1+2\pi)$

Same encoding

Different positions

Consequence in Neural Networks

The model receives:

Word1 $\rightarrow \sin(1)$

Word2 $\rightarrow \sin(1)$

Model thinks:

These words are at the **same position**

But in reality:

They are **far apart in the sequence**

3. Why This Is a Serious Issue

1. Loss of Uniqueness

- Each position must have a **distinct representation**
- Sine alone cannot guarantee this

2. Ambiguity in Long Sequences

- In long texts (e.g., documents, PDFs), repetitions increase
- Many positions collapse to same values

3. Misleading the Model

- Self-attention relies on positional signals
- Duplicate encodings → incorrect attention patterns

4. Key Insight from This Problem

We need a positional encoding where:

- Positions do **not collide**
- Representations are **distinct**
- Periodicity is **controlled**, not harmful

5. Solution: Combine Sine with Cosine

Instead of:

$$PE(pos) = \sin(pos)$$

We use **two functions**:

- Sine
- Cosine

6. Why Cosine Helps

Phase Difference

- Sine and cosine are **out of phase**
- Even if sine values match, cosine values differ

Example Intuition

For two positions:

Position A $\rightarrow (\sin(A), \cos(A))$

Position B $\rightarrow (\sin(B), \cos(B))$

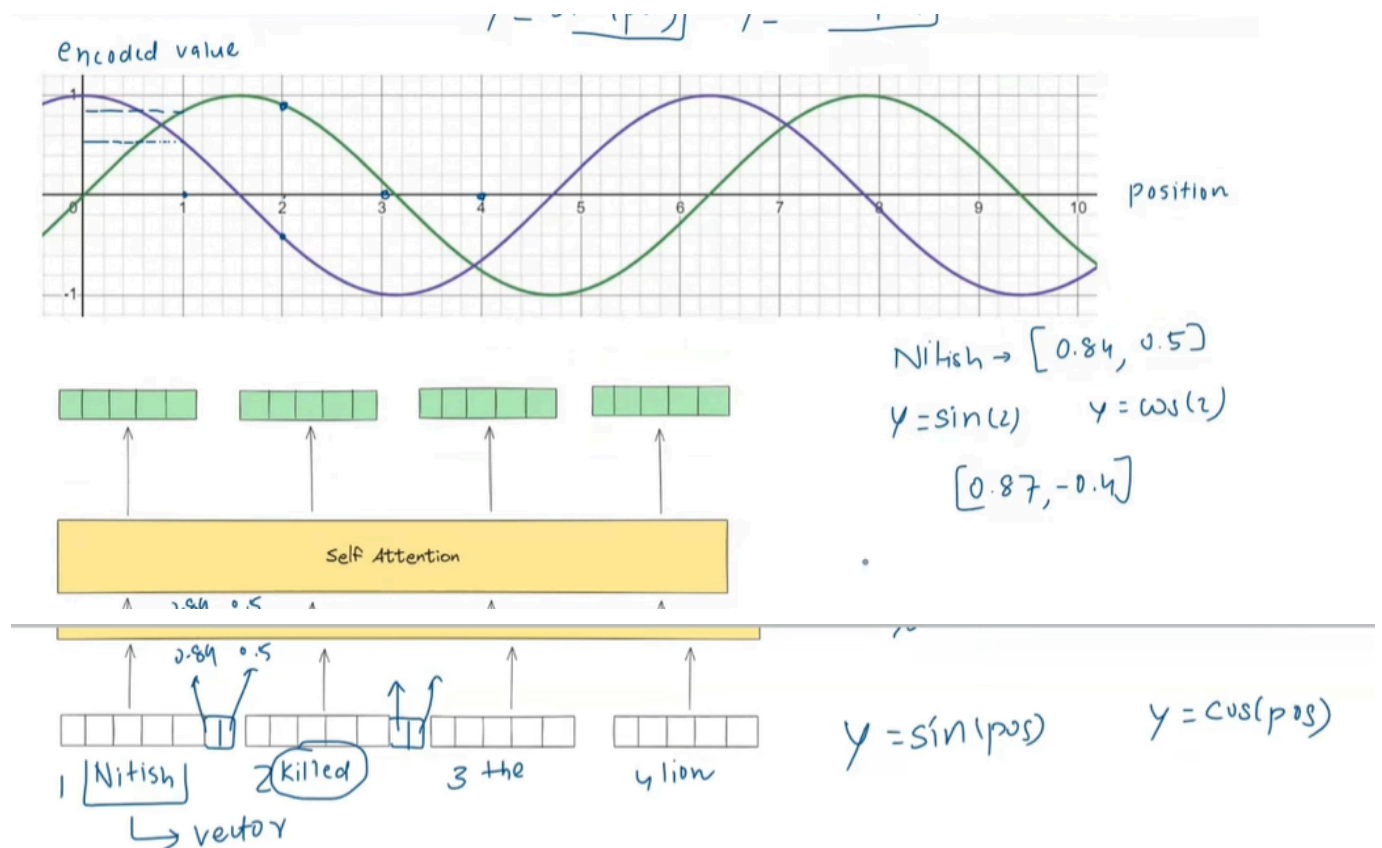
Even if:

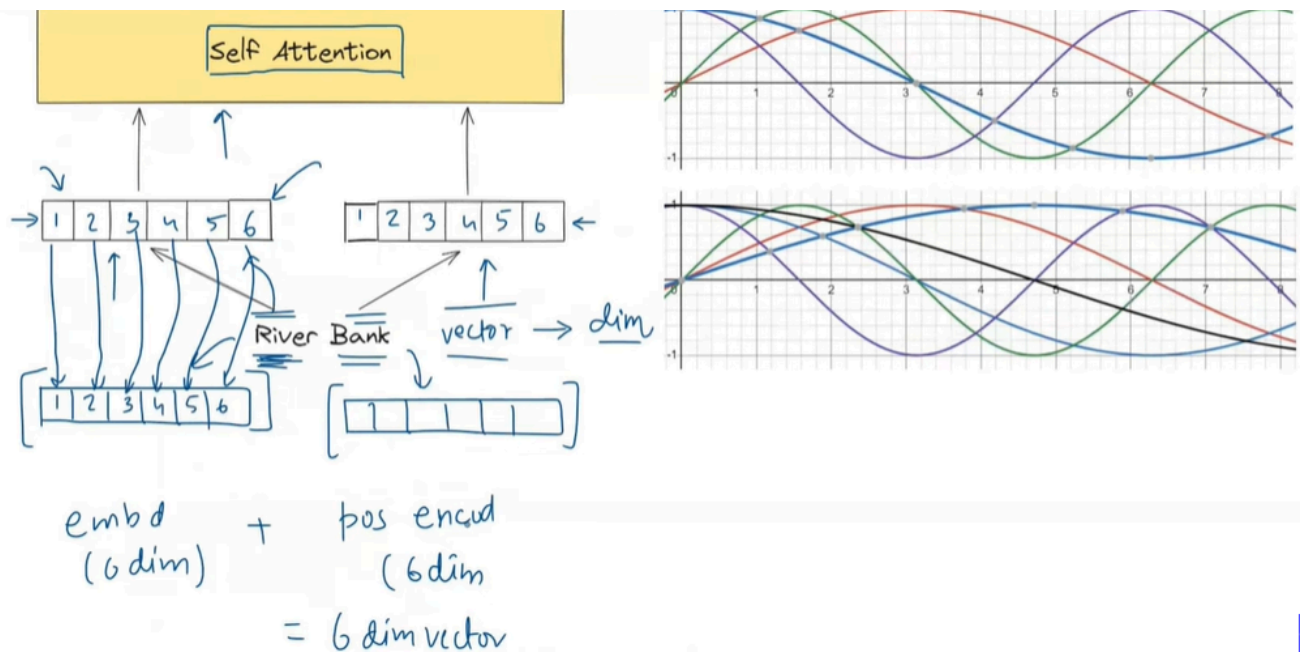
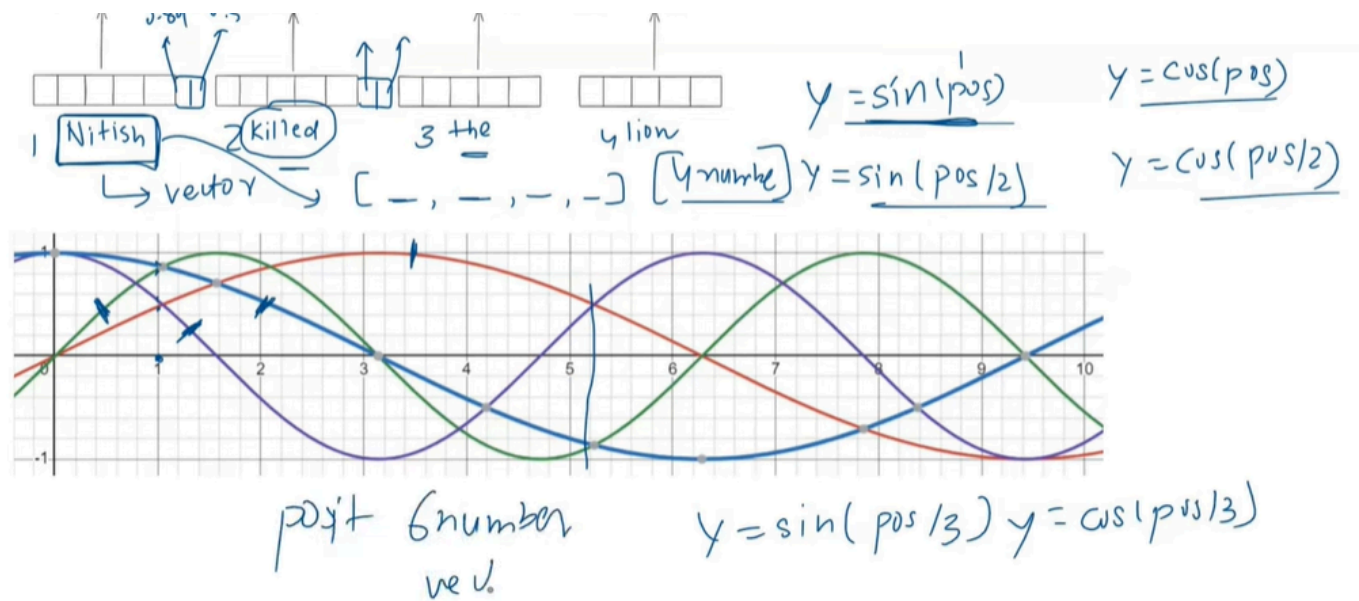
$$\sin(A) = \sin(B)$$

It is **very unlikely** that:

$$\cos(A) = \cos(B)$$

✓ Together $\rightarrow$ **unique pair**





Why Do We Add Positional Encoding Instead of Concatenating?

1. Setup

We have:

- **Word Embedding** → captures meaning
- **Positional Encoding** → captures order

Both are vectors of same dimension:

$\text{Embedding} \in \mathbb{R}^{d_{\text{model}}}$

2. Two Possible Approaches

Option 1: Concatenation

[Embedding || Position]

Dimension becomes:

$2 \times d_{\text{model}}$

Option 2: Addition (Used in Transformers)

Final Input = Embedding + Positional

Dimension remains:

d_{model}

3. Why Concatenation is NOT Used

1. Increases Model Size

- If dimension doubles:
 - All weight matrices in attention layers also grow
- Leads to:
 - More parameters
 - More memory usage

2. Slower Training

- Larger vectors $\rightarrow$ more computations
- Attention complexity increases

Result:

- Higher training time
- Less efficiency

3. Overkill Representation

- Model now has to **learn how to combine**:

- Meaning (embedding)
- Position (encoding)

This adds unnecessary learning burden

4. Why Addition Works Better

1. Same Dimension → Efficient

- No increase in parameters
- Keeps architecture simple

2. Blends Meaning + Position Naturally

Addition creates:

Input = Semantic Info + Positional Info

Each dimension now contains:

A mix of **what the word is** + **where it is**

3. Model Can Still Separate Them

This is subtle but very important:

- Positional encoding follows a **known pattern** (sin/cos)
- Word embeddings are **learned**

Neural network can:

- Learn to distinguish patterns
- Extract position vs meaning when needed

4. Empirically Works (Very Important)

From **Attention Is All You Need**:

- Addition was tested
- It worked **efficiently and accurately**

No need for more complex alternatives

5. Intuition (Very Important for Understanding)

Think of it like:

Audio Analogy

- Embedding = **music**
- Position encoding = **background tone**

Adding them:

- You still hear both
- Brain can separate them

6. Key Insight

Addition works because neural networks don't need explicit separation — they can **learn to disentangle combined signals**.

7. Final Summary (Exam Ready)

Concatenation:

- Increases dimension
- More parameters
- Slower training

Addition:

- Keeps dimension fixed
- Efficient
- Blends semantic + positional info

Therefore:

Transformers use **addition** to combine embeddings and positional encoding for efficiency and effective learning.

One-Line Intuition

Instead of giving position and meaning separately, we **merge them into a single vector**, and let the model learn how to use both.

3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [9].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .

We also experimented with using learned positional embeddings [9] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

◆ 7. Final Transformer Encoding

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

pos → position
pos=0 pos=1

$d_{\text{model}} \rightarrow \text{dim of embedding}$
 $d_{\text{model}}=6$

$i = 0 - \lfloor d_{\text{model}}/2 \rfloor = 0 - \lfloor 6/2 \rfloor = 0 - 3 = -3$
for $i=0$ $0-2$

for $i=0 \rightarrow pos=0$

$PE(0, 0) = \sin(0/10000^0) = 0$

$PE(0, 1) = \cos(0/10000^0) = 1$

for $i=1 \rightarrow pos=1$

$PE(1, 0) = \sin(1/10000^0) = 0.84$

$PE(1, 1) = \cos(1/10000^0) = 0.54$

Diagram illustrating the positional encoding process:

- Input sequence: "River", "bank"
- Positions: pos=0 for "River", pos=1 for "bank"
- Embedding dimensions: 1, 2, 3, 4, 5, 6
- Positional encodings are added to the embeddings at the bottom of the encoder and decoder stacks.

$$PE(0,2) = \sin(0/10000^{1/3}) = 0$$

$$PE(0,3) = \cos(0/10000^{1/3}) = 1$$

$$PE(1,2) = \sin(1/10000^{1/3}) = 0.00$$

$$PE(1,3) = \cos(1/10000^{1/3}) = 0.99$$

for i=2 ✓

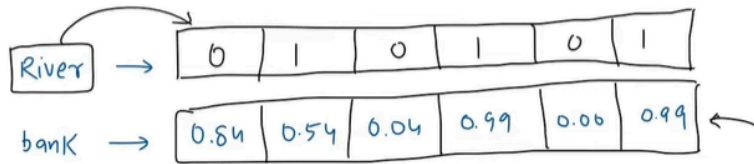
$$PE(0,4) = \sin(0/10000^{2/3}) = 0$$

$$PE(0,5) = \cos(0/10000^{2/3}) = 1$$

for i=2

$$PE(1,4) = \sin(1/10000^{2/3}) = 0.00$$

$$PE(1,5) = \cos(1/10000^{2/3}) = 0.99$$



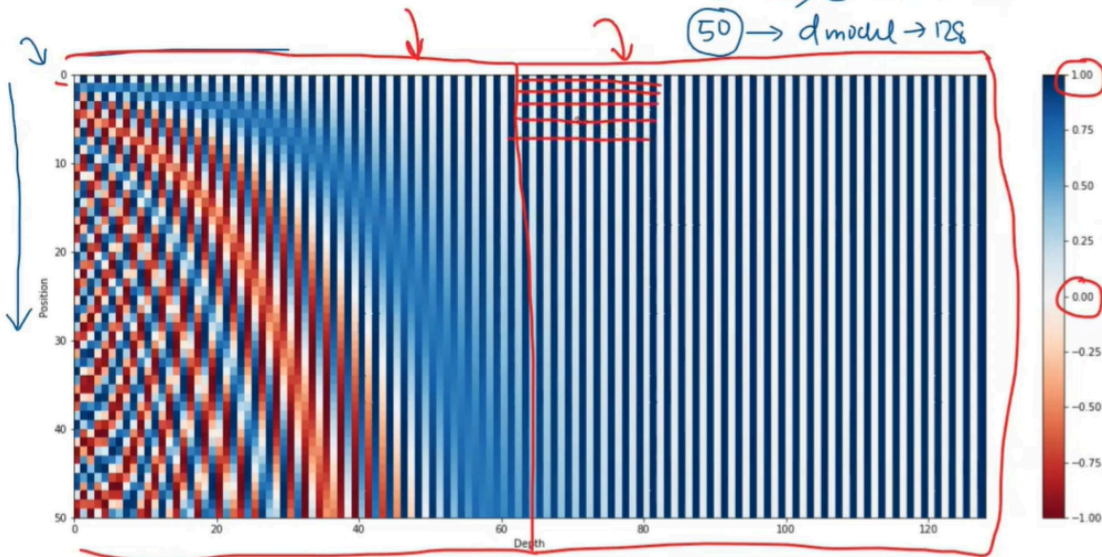
Interesting Observations

24 May 2024 02:06

Sentence → 50 words

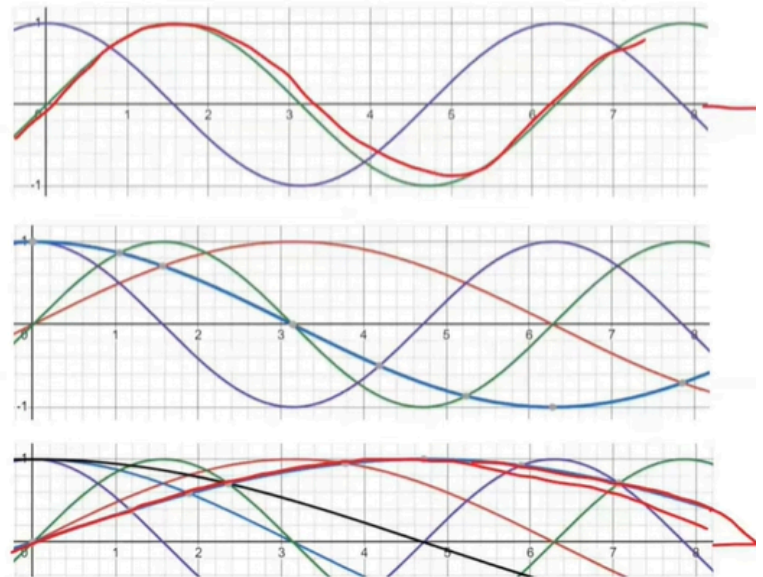
↓ embedding → 128 $d_{model} = 128$

(50) → $d_{model} \rightarrow 128$



100 binary encoding 4bit 8bit

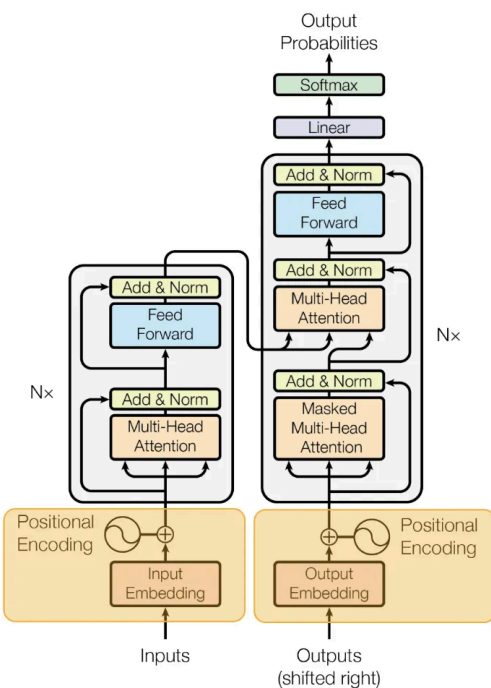
0:	0 0 0 0	8:	1 0 0 0
1:	0 0 0 1	9:	1 0 0 1
2:	0 0 1 0	10:	1 0 1 0
3:	0 0 1 1	11:	1 0 1 1
4:	0 1 0 0	12:	1 1 0 0
5:	0 1 0 1	13:	1 1 0 1
6:	0 1 1 0	14:	1 1 1 0
7:	0 1 1 1	15:	1 1 1 1



Formulation for Positional Encoding

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$



$\frac{e_1 + n}{\cos(\frac{n}{10000^{2/5}})}$	$\frac{e_2 + n}{\sin(\frac{n}{10000^{4/5}})}$	$\frac{e_3 + n}{\cos(\frac{n}{10000^{6/5}})}$	$\frac{e_4 + n}{\sin(\frac{n}{10000^{8/5}})}$	$\frac{e_5 + n}{\cos(\frac{n}{10000^{10/5}})}$
---	---	---	---	--

Input Encoding for Token n to the Transformer

© AML.com Research

$\frac{n}{\cos(\frac{n}{10000^{2/5}})}$	$\frac{n}{\sin(\frac{n}{10000^{4/5}})}$	$\frac{n}{\cos(\frac{n}{10000^{6/5}})}$	$\frac{n}{\sin(\frac{n}{10000^{8/5}})}$	$\frac{n}{\cos(\frac{n}{10000^{10/5}})}$
$i=1$	$i=2$	$i=3$	$i=4$	$i=5$

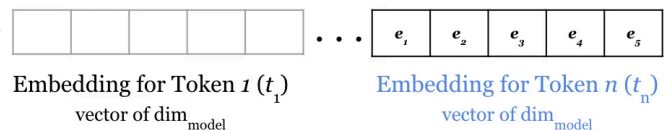
Positional Encoding for Token at position n

where, $pos=n$, $d_{model}=5$

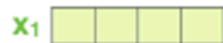


element-wise Addition

Input Token Embeddings

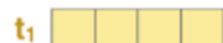


EMBEDDING
WITH TIME
SIGNAL



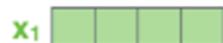
=

POSITIONAL
ENCODING



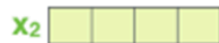
+

EMBEDDINGS

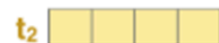


INPUT

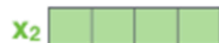
Je



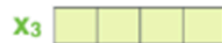
=



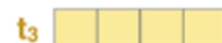
+



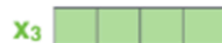
suis



=



+



étudiant